

ARSENAL TECH

L'ÉLITE 2026

PARTIE III · LE SOFTWARE ENGINEERING DE HAUTE PRÉCISION

CHAPITRE 5

Clean Code Militaire & Patterns de Domination

SOLID · DRY · GoF Patterns · Refactoring de Legacy Code

~\$3.6T / an (CISQ 2025) Coût de la dette technique	~40% en moyenne prod Bugs évités par SOLID	+60% équipe 6 mois Vélocité après refactoring	23 patterns fondateurs Design Patterns GoF référencés
--	---	--	--

Introduction : L'Artisanat comme Discipline Militaire

Il existe une vérité inconfortable dans le monde du software engineering : la grande majorité du code écrit en entreprise est du code survivant, pas du code conçu. Il fonctionne, en quelque sorte, dans certaines conditions, jusqu'à ce qu'il ne fonctionne plus. Le Clean Code Militaire est la philosophie selon laquelle chaque ligne de code doit être écrite comme si elle allait être lue, modifiée et étendue par un professionnel exigeant sous pression maximale — parce que c'est exactement ce qui va se passer.

SOLID et DRY ne sont pas des dogmes académiques. Ce sont des heuristiques de survie pour les systèmes qui vivent 5, 10, 15 ans en production. Les Design Patterns GoF ne sont pas des solutions à mémoriser — ce sont des vocabulaires partagés qui permettent à une équipe de communiquer en quelques mots ce qui prendrait des paragraphes à décrire. Et le refactoring de legacy code n'est pas un luxe réservé aux moments calmes — c'est la compétence qui distingue l'ingénieur senior de l'ingénieur moyen.

► Sous-Partie 1 : SOLID & DRY — L'Application Brutale en Entreprise

◆ Pourquoi SOLID Échoue en Entreprise (et Comment l'Appliquer Vraiment)

SOLID est enseigné avec des exemples d'animaux qui font des sons ou de formes géométriques qui calculent des aires. Ces exemples sont inutiles. En entreprise, SOLID s'applique sur des systèmes de paiement à 200K lignes, des moteurs de règles métier, des APIs publiques avec des centaines de clients. La traduction brutale de SOLID dans ce contexte est ce que ce chapitre vous donne.

◆ S — Single Responsibility Principle : La Règle des 100 Lignes

Une classe ne doit avoir qu'une seule raison de changer. En pratique : si vous ne pouvez pas décrire ce que fait votre classe en une phrase sans utiliser "et", elle viole le SRP. Voici la violation la plus commune et sa correction brutale :

* JAVA — SRP : DE LA GOD CLASS AU SERVICE LAYER

```
// == VIOLATION SRP : La classe fait TOUT ==
// God Class — 800 lignes, impossible à tester, impossible à modifier
public class UserManager {
    public User createUser(UserDTO dto) { /* validation */ }
    public void sendWelcomeEmail(User user) { /* SMTP */ } // ← SMTP ici ?
    public void logUserCreation(User user) { /* logging */ } // ← Logging ici ?
    public void updateDatabaseSchema() { /* migration */ } // ← Migration ici ??
    public UserDTO toDTO(User user) { /* mapping */ }
    public boolean validatePassword(String pwd) { /* règles */ }
    public void generateReport(List<User> users) { /* PDF */ } // ← PDF ici ???
}
// Raisons de changer : règles validation, format email, config SMTP,
//                      schema DB, format PDF, règles métier...
// = 7 raisons de changer → 7 fois plus probable de casser quelque chose

// == APRÈS SRP : Chaque classe = 1 responsabilité = 1 raison de changer =

// Domaine pur — ne connaît NI email NI logging NI base de données
public class UserCreationService {
    private final UserRepository repository;
    private final PasswordPolicy passwordPolicy;

    public User createUser(CreateUserCommand cmd) {
        passwordPolicy.validate(cmd.password()); // délègue la règle
        User user = User.create(cmd);           // factory method
        return repository.save(user);           // délègue la persistance
    }
}

// Infrastructure : Gère l'email — rien d'autre
public class WelcomeEmailSender {
    private final EmailTemplate template;
    private final SmtpClient smtp;

    public void send(User user) {
        Email email = template.renderFor(user);
        smtp.send(email);
    }
}

// Application : Orchestration — délègue à tout le monde, décide de rien
@Service
public class RegisterUserUseCase {
    private final UserCreationService creationService;
    private final WelcomeEmailSender emailSender;
    private final AuditLogger auditLogger;

    public UserDTO execute(CreateUserCommand cmd) {
        User user = creationService.createUser(cmd);
        emailSender.send(user);
        auditLogger.log(UserCreatedEvent.from(user));
        return UserMapper.toDTO(user);
    }
}

// Résultat : 3 classes de 40 lignes chacune. Testables isolément.
// Modifier l'email ne touche jamais la logique métier.
```

◆ O — Open/Closed Principle : L'Extension sans Modification

Ouvert à l'extension, fermé à la modification. Le test concret : si ajouter un nouveau type de paiement nécessite de modifier une classe existante, vous violez l'OCP — et vous créez un risque régressionnel à chaque évolution métier.

♦ JAVA — OCP : STRATEGY PATTERN + AUTO-REGISTRATION

```
// == VIOLATION OCP : Chaque nouveau type = modification de la classe =
public class PaymentProcessor {
    public void process(Payment payment) {
        if (payment.type().equals("WAVE")) {
            waveGateway.charge(payment);
        } else if (payment.type().equals("ORANGE_MONEY")) {
            orangeGateway.transfer(payment);
        } else if (payment.type().equals("CARD")) {
            cardGateway.authorize(payment);
        }
        // ← Ajouter MTN_MOMO = modifier cette classe = risque de régression
    }
}

// == APRÈS OCP : Stratégie + Registry = extensible sans modification =

// Contrat d'extension — stable, ne change jamais
public interface PaymentGateway {
    boolean supports(PaymentMethod method);
    PaymentResult process(Payment payment);
}

// Implémentation Wave — complètement indépendante
@Component
public class WaveGateway implements PaymentGateway {
    public boolean supports(PaymentMethod m) { return m == WAVE; }
    public PaymentResult process(Payment p) { return waveApi.charge(p); }
}

// Implémentation MTN MoMo — ajoutée sans toucher quoi que ce soit
@Component
public class MtnMomoGateway implements PaymentGateway {
    public boolean supports(PaymentMethod m) { return m == MTN_MOMO; }
    public PaymentResult process(Payment p) { return mtnApi.requestPayment(p); }
}

// Orchestrateur : FERMÉ à la modification — injecte tous les gateways
@Service
public class PaymentProcessor {
    private final List<PaymentGateway> gateways; // Injection par Spring

    public PaymentResult process(Payment payment) {
        return gateways.stream()
            .filter(g -> g.supports(payment.method()))
            .findFirst()
            .orElseThrow(() -> new UnsupportedOperationException(payment.method()))
            .process(payment);
    }
}

// Ajouter MTN_MOMO : Créer MtnMomoGateway, annoter @Component.
// Zéro modification de code existant. Zéro risque de régression.
```

◆ L, I, D — Les Trois Principes Complémentaires

Principe	Énoncé Brutal	Test de Violation	Correction Type
Liskov (L)	Remplacer une classe par une sous-classe ne doit pas casser le programme	La sous-classe lève une exception que la base ne lève pas	Composition plutôt qu'héritage. Rectangle/Carré est l'anti-pattern classique
Interface Segregation (I)	Ne pas forcer une classe à implémenter des méthodes qu'elle n'utilise pas	Méthode implémentée avec throw UnsupportedOperationException()	Diviser les interfaces larges en interfaces spécifiques par rôle
Dependency Inversion (D)	Dépendre d'abstractions, jamais d'implémentations concrètes	new MongoRepository() dans une classe de service — couplage dur	Injecter l'interface UserRepository, pas MongoUserRepository

◆ DRY — Don't Repeat Yourself : La Discipline de l'Abstraction Juste

DRY ne signifie pas "n'écris jamais deux fois du code similaire". Cette interprétation mène à une sur-abstraction catastrophique — la fameuse Wrong Abstraction (Sandi Metz). DRY signifie qu'il ne doit pas y avoir deux endroits dans le code qui représentent la même CONNAISSANCE du domaine. La duplication de code est parfois préférable à la mauvaise abstraction.

```
* JAVA — DRY : SOURCE DE VÉRITÉ UNIQUE VS WRONG ABSTRACTION

// == FAUSSE APPLICATION DRY : Abstraction prématurée ==
// Code SIMILAIRE mais pas MÊME CONNAISSANCE — NE PAS abstraire
public BigDecimal calculateEmployeeBonus(Employee emp) {
    return emp.getSalary().multiply(BigDecimal.valueOf(0.1));
}
public BigDecimal calculateClientDiscount(Client client) {
    return client.getPurchaseAmount().multiply(BigDecimal.valueOf(0.1));
}
// ← Même calcul (×0.1) mais connaissances DIFFÉRENTES.
// Si la règle bonus change → le discount ne doit PAS changer.
// Abstraire serait une ERREUR — c'est la Wrong Abstraction.

// == VRAIE VIOLATION DRY : Même connaissance en deux endroits ==
// La règle : Un virement est plafonné à 5 000 000 XOF

// Dans le service de paiement mobile :
public void processMobilePayment(Payment p) {
    if (p.amount().compareTo(new BigDecimal("5000000")) > 0) {
        throw new LimitExceededException(); // ← Connaissance du domaine
    }
}
// Dans le service de virement bancaire :
public void processBankTransfer(Transfer t) {
    if (t.amount().compareTo(new BigDecimal("5000000")) > 0) { // ← DUPLIQUÉ
        throw new TransferLimitException(); // Même règle, endroit différent
    }
}
// Quand le plafond passe à 10M XOF (décret BCEAO) →
// Modifier 1 endroit, oublier l'autre → BUG de production silencieux.

// == CORRECTION DRY : Source de vérité unique pour la règle ==
// Value Object qui encapsule la connaissance du plafond
public record TransferAmount(BigDecimal value) {
```

```
private static final BigDecimal MAX_AMOUNT = new BigDecimal("5000000");

public TransferAmount { // Validation dans le constructeur
    if (value.compareTo(BigDecimal.ZERO) <= 0)
        throw new IllegalArgumentException("Amount must be positive");
    if (value.compareTo(MAX_AMOUNT) > 0)
        throw new TransferLimitExceededException(
            "Transfer limit is " + MAX_AMOUNT + " XOF");
}

// Maintenant : mobile payment, bank transfer, tous créent TransferAmount.
// Changer MAX_AMOUNT = changer 1 seule ligne. Partout. Toujours correct.

// == DRY appliqué aux configurations (Pattern : Central Config) ==
// MAUVAIS : Valeurs magiques éparpillées
// kafka.producer.batch.size = 65536 (dans kafka.properties)
// batch_size = 64 * 1024 (dans le code Python)
// batchSize: 65536 (dans le config.yaml)

// BON : Source unique, générée automatiquement pour chaque langage
@ConfigurationProperties(prefix = "payment")
public record PaymentConfig(
    BigDecimal maxTransferAmount, // 5000000
    int maxRetryAttempts, // 3
    Duration transactionTimeout // PT30S
) {}

// Exportée via Config Server → consommée par tous les services.
```

► Sous-Partie 2 : Design Patterns GoF — Factory, Strategy, Observer

◆ Les 23 Patterns GoF : Carte d'Orientation

En 1994, Gamma, Helm, Johnson et Vlissides publient "Design Patterns: Elements of Reusable Object-Oriented Software" — le Gang of Four (GoF). Trente ans plus tard, ces 23 patterns restent le vocabulaire universel de l'ingénierie logicielle. Ce chapitre maîtrise en profondeur les trois patterns les plus déployés en production moderne : Factory Method, Strategy et Observer.

LES 23 PATTERNS GoF : CARTE COMPLÈTE

CRÉATIONNELS (5) — Comment créer des objets

- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton (à éviter en microservices)

STRUCTURELS (7) — Comment composer des objets

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

COMPORTEMENTAUX (11) — Comment les objets communiquent

- Chain of Resp.
- Command
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor
- Interpreter

PATTERNS ÉTUDIÉS EN PROFONDEUR CE CHAPITRE :

- ★ Factory Method (Créationnel — le plus universel)
- ★ Strategy (Comportemental — remplace tous les if/switch)
- ★ Observer (Comportemental — backbone des systèmes event-driven)

◆ Factory Method : L'Art de Créer sans Connaître

Le Factory Method définit une interface pour créer un objet, mais laisse les sous-classes décider quelle classe instancier. En 2026, son usage le plus critique est dans les systèmes multi-canal : créer le bon type de notification, le bon gateway de paiement, le bon connecteur de données sans que le code client ne connaisse l'implémentation concrète.

FACTORY METHOD PATTERN

Client (ne connaît que l'interface)

```

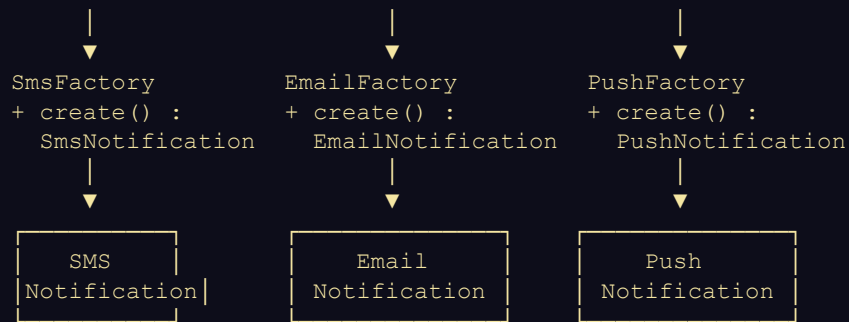
|
| notificationFactory.create(channel)
|
▼

```

```

NotificationFactory (interface)
+ create(channel) : Notification

```



Client ne fait que : `factory.create(channel).send(message)`
 Zero couplage avec `SmsNotification`, `EmailNotification`, etc.

* JAVA — FACTORY METHOD : SYSTÈME DE NOTIFICATION MULTI-CANAL

```
// — FACTORY METHOD : Système de notification multi-canal —

// — Produit : Interface commune —
public interface Notification {
    NotificationResult send(NotificationMessage message, Recipient recipient);
    NotificationChannel channel();
}

// — Produits concrets —
@Component
public class SmsNotification implements Notification {
    private final SmsProvider smsProvider;

    public NotificationResult send(NotificationMessage msg, Recipient r) {
        SmsRequest req = SmsRequest.of(r.phoneNumber(), msg.text());
        return smsProvider.send(req).toResult();
    }

    public NotificationChannel channel() { return NotificationChannel.SMS; }
}

@Component
public class WhatsAppNotification implements Notification {
    private final WhatsAppBusinessApi api;

    public NotificationResult send(NotificationMessage msg, Recipient r) {
        // Template WhatsApp Business — requis pour Afrique (Meta policy)
        WhatsAppTemplate template = WhatsAppTemplate.builder()
            .name(msg.templateName())
            .language(r.preferredLanguage())
            .parameters(msg.variables())
            .build();
        return api.sendTemplate(r.whatsappId(), template).toResult();
    }

    public NotificationChannel channel() { return NotificationChannel.WHATSAPP; }
}

// — Factory : Crée le bon canal selon le contexte —
@Service
public class NotificationFactory {
    private final Map<NotificationChannel, Notification> providers;

    // Spring injecte TOUS les beans Notification — OCP respecté
}
```



```

public NotificationFactory(List<Notification> notifications) {
    this.providers = notifications.stream()
        .collect(Collectors.toMap(Notification::channel, n -> n));
}

public Notification create(NotificationChannel channel) {
    return Optional.ofNullable(providers.get(channel))
        .orElseThrow(() -> new UnsupportedOperationException(channel));
}

// Factory intelligente : Choisit le canal selon préférence + disponibilité
public Notification createOptimal(Recipient recipient) {
    return recipient.preferredChannels().stream()
        .filter(providers::containsKey)
        .map(providers::get)
        .findFirst()
        .orElse(providers.get(NotificationChannel.SMS)); // Fallback SMS
}

// — Client : Ne connaît QUE l'interface —
@Service
public class PaymentConfirmationService {
    private final NotificationFactory notificationFactory;

    public void confirmPayment(Payment payment, Recipient recipient) {
        NotificationMessage msg = NotificationMessage.paymentConfirmation(payment);
        // Client ne sait pas si c'est SMS, WhatsApp, Push...
        notificationFactory.createOptimal(recipient).send(msg, recipient);
    }
}

// Ajouter Telegram, Signal ou MoMo SMS : créer la classe, annoter @Component.
// Le client PaymentConfirmationService n'est JAMAIS modifié.

```

◆ Strategy Pattern : Remplacer Tous Vos if/switch par de l'Élégance

Le Strategy Pattern définit une famille d'algorithmes interchangeables, les encapsule individuellement et les rend substituables. Son application la plus percutante en production : les moteurs de calcul (frais, scoring, tarification) où les règles varient par contexte et changent fréquemment sans devoir toucher au code client.

* JAVA — STRATEGY PATTERN : MOTEUR DE CALCUL DE FRAIS FINTECH

```

// — STRATEGY : Moteur de calcul des frais de virement —
// Règles métier : Frais différents selon montant, devise, corridor géo

// — Contexte —
public record TransferContext(
    BigDecimal amount,
    Currency sourceCurrency,
    String sourceCountry,
    String destinationCountry,
    boolean isPremiumUser
) {}

// — Interface Strategy —
public interface FeeCalculationStrategy {
    boolean appliesTo(TransferContext ctx); // Condition d'éligibilité
    FeeResult calculate(TransferContext ctx); // Calcul effectif
}

```

```

    int priority(); // Ordre d'évaluation
}

// — Stratégie 1 : Virements intra-UEMOA (tarif préférentiel BCEAO) —
@Component
public class UEMOAInternalFeeStrategy implements FeeCalculationStrategy {
    private static final Set<String> UEMOA_COUNTRIES =
        Set.of("SN", "CI", "ML", "BF", "TG", "BJ", "NE", "GW");

    public boolean appliesTo(TransferContext ctx) {
        return UEMOA_COUNTRIES.contains(ctx.sourceCountry())
            && UEMOA_COUNTRIES.contains(ctx.destinationCountry());
    }

    public FeeResult calculate(TransferContext ctx) {
        // Directive BCEAO 2024 : 0.35% intra-UEMOA, max 10 000 XOF
        BigDecimal fee = ctx.amount().multiply(BigDecimal.valueOf(0.0035));
        BigDecimal cappedFee = fee.min(new BigDecimal("10000"));
        return new FeeResult(cappedFee, "XOF", "UEMOA_CORRIDOR");
    }

    public int priority() { return 10; } // Priorité haute — appliquée en premier
}

// — Stratégie 2 : Utilisateurs Premium —
@Component
public class PremiumUserFeeStrategy implements FeeCalculationStrategy {
    public boolean appliesTo(TransferContext ctx) {
        return ctx.isPremiumUser() && ctx.amount().compareTo(new BigDecimal("100000")) >
0;
    }

    public FeeResult calculate(TransferContext ctx) {
        // Premium > 100K XOF : frais fixes 2000 XOF seulement
        return new FeeResult(new BigDecimal("2000"), "XOF", "PREMIUM_FLAT");
    }

    public int priority() { return 5; } // Priorité supérieure à standard
}

// — Stratégie 3 : Standard international (fallback) —
@Component
public class StandardInternationalFeeStrategy implements FeeCalculationStrategy {
    public boolean appliesTo(TransferContext ctx) { return true; } // Toujours applicable
    public FeeResult calculate(TransferContext ctx) {
        BigDecimal fee = ctx.amount().multiply(BigDecimal.valueOf(0.015)) // 1.5%
            .add(new BigDecimal("5000")); // + 5000 XOF fixe
        return new FeeResult(fee, "XOF", "STANDARD_INTERNATIONAL");
    }

    public int priority() { return 1; } // Priorité basse — fallback
}

// — Context : Sélectionne et applique la bonne stratégie —
@Service
public class FeeCalculationEngine {
    private final List<FeeCalculationStrategy> strategies;

    public FeeCalculationEngine(List<FeeCalculationStrategy> strategies) {
        // Trier par priorité descendante au démarrage — une fois
        this.strategies = strategies.stream()
            .sorted(Comparator.comparingInt(FeeCalculationStrategy::priority).reversed())
            .toList();
    }
}

```

```

public FeeResult calculateFee(TransferContext ctx) {
    return strategies.stream()
        .filter(s -> s.appliesTo(ctx))
        .findFirst()    // Prend la stratégie de plus haute priorité applicable
        .orElseThrow() // Ne peut pas arriver — StandardInternational est toujours
vrai
        .calculate(ctx);
    }
}
// Ajouter un corridor Mali-Côte d'Ivoire spécial : 1 nouvelle classe.
// Zéro modification du code existant. Tests unitaires par stratégie.

```

◆ Observer Pattern : L'Architecture Événementielle à l'Échelle Objet

L'Observer (ou Pub/Sub interne) définit une dépendance un-à-plusieurs entre objets : quand un objet change d'état, tous ses dépendants sont notifiés automatiquement. En 2026, l'Observer est le pattern qui permet de découpler les effets de bord (emails, logs, analytics) de la logique métier core — sans Kafka, sans infrastructure externe, au niveau de l'application.

♦ JAVA / SPRING — OBSERVER AVEC DOMAIN EVENTS DÉCOUPLÉS

```

// — OBSERVER PATTERN : Événements de domaine découplés —————
// Scénario : Transaction validée → email + analytics + fraud check + audit

// — Domain Event : Ce qui s'est passé —————
public record TransactionValidatedEvent(
    String      transactionId,
    String      userId,
    BigDecimal   amount,
    String      currency,
    String      merchantId,
    Instant     occurredAt
) implements DomainEvent {}

// — Sujet (Subject / Observable) —————
// Dans Spring : ApplicationEventPublisher joue ce rôle
@Service
public class TransactionService {
    private final TransactionRepository repository;
    private final ApplicationEventPublisher eventPublisher;

    @Transactional
    public Transaction validateTransaction(ValidateTransactionCommand cmd) {
        Transaction txn = repository.findById(cmd.transactionId())
            .orElseThrow(TransactionNotFoundException::new);

        txn.validate(); // Logique métier pure — rien d'autre
        repository.save(txn);

        // Publication de l'événement — ne connaît PAS les observateurs
        eventPublisher.publishEvent(TransactionValidatedEvent.from(txn));

        return txn; // Le service ne sait pas que 4 systèmes vont réagir
    }
}

// — Observer 1 : Confirmation email —————

```

```

@Component
public class TransactionConfirmationEmailObserver {
    @EventListener
    @Async // Non-bloquant - n'impacte pas la latence de la transaction
    public void onTransactionValidated(TransactionValidatedEvent event) {
        emailService.sendTransactionConfirmation(
            event.userId(), event.transactionId(), event.amount());
    }
}

// — Observer 2 : Détection de fraude —————
@Component
public class FraudDetectionObserver {
    @EventListener
    @Async
    public void onTransactionValidated(TransactionValidatedEvent event) {
        FraudScore score = fraudEngine.analyze(event);
        if (score.isHighRisk()) {
            alertService.flagForReview(event.transactionId(), score);
        }
    }
}

// — Observer 3 : Analytics temps réel —————
@Component
public class AnalyticsObserver {
    @EventListener
    @Async
    public void onTransactionValidated(TransactionValidatedEvent event) {
        analyticsService.track('transaction.validated', Map.of(
            'amount', event.amount(),
            'currency', event.currency(),
            'merchant', event.merchantId()
        ));
    }
}

// — Observer 4 : Audit log immuable —————
@Component
public class AuditObserver {
    @EventListener // Synchrones - audit doit être dans la même transaction
    @TransactionalEventListener(phase = AFTER_COMMIT)
    public void onTransactionValidated(TransactionValidatedEvent event) {
        auditLogger.logImmutable('TRANSACTION_VALIDATED', event);
    }
}

// TransactionService : 0 connaissance de email, fraude, analytics, audit.
// Ajouter un nouvel observateur : créer la classe, annoter @EventListener.
// Supprimer un observateur : supprimer la classe. Service inchangé.

```

► **Sous-Partie 3 : Refactoring de Legacy Code — Transformer l'Obsolète en Élite**

◆ **La Psychologie du Legacy : Comprendre avant d'Agir**

Le legacy code n'est pas du mauvais code. C'est du code qui a survécu — ce qui signifie qu'à un moment, il fonctionnait assez bien pour être mis en production et y rester. Michael Feathers dans "Working Effectively with Legacy Code" (2004) donne la définition la plus opérationnelle : du code legacy est du code sans tests. Cette définition change tout — elle transforme le refactoring d'une question esthétique en une question d'ingénierie mesurable.

● **TAXONOMIE DU LEGACY CODE — DIAGNOSTIC EN 5 MINUTES**

- NIVEAU 1 — Code Ancien Propre : Fonctionne, style daté, mais tests présents. → Moderniser progressivement, pas urgent.
- NIVEAU 2 — Code Vivant Couplé : Fonctionne, pas de tests, mais logique claire. → Ajouter des tests, puis refactorer.
- NIVEAU 3 — Code Spaghetti Actif : Fonctionne parfois, dépendances circulaires, tests impossibles. → Strangler Fig Pattern.
- NIVEAU 4 — Big Ball of Mud : Fonctionne par miracle, personne ne comprend, peur de toucher. → Réécriture progressive obligatoire.
- NIVEAU 5 — Code Zombie : Ne fonctionne plus vraiment, patches sur patches. → Arrêter de patcher, réécrire ou retirer.
- RÈGLE D'OR : Ajouter des tests AVANT de refactorer. Sans filet de sécurité, refactorer = risque maximal.

◆ **Les 6 Techniques de Refactoring Indispensables**

Technique	Problème Résolu	Risque	Quand l'Utiliser
Extract Method	Méthode de 200 lignes illisible	Très faible	En permanence — reflex quotidien
Extract Class	God Class avec 20 responsabilités	Faible	Quand SRP est violé
Move Method	Méthode dans la mauvaise classe	Faible	Feature Envy détecté
Replace Conditional with Polymorphism	switch/if interminable sur types	Modéré	Strategy Pattern applicable
Introduce Parameter Object	Méthode à 7+ paramètres	Très faible	Dès 4+ paramètres liés
Strangler Fig	Remplacement d'un module entier	Élevé	Réécriture progressive legacy

◆ **Cas Pratique Complet : Refactoring d'un Système de Paiement Legacy**

Voici un cas réel (anonymisé) : une API de paiement PHP legacy de 2011, migrée vers Java en 2017 sans refactoring, et qui maintenant doit être modernisée sans interruption de service. Nous appliquons la méthode en 4 phases.

* JAVA — REFACTORING LEGACY EN 4 PHASES AVEC STRANGLER FIG

```
// =====
// PHASE 1 : DIAGNOSTIC — Identifier les seams (coutures)
// Un seam est un endroit où vous pouvez changer le comportement
// sans modifier le code lui-même. C'est là qu'on greffe les tests.
// =====

// CODE LEGACY (simplifié mais représentatif)
public class PaymentProcessor { // 800 lignes, 0 test
    public String processPayment(String userId, double amount,
                                String type, String currency,
                                boolean express, int retries) {

        // Validation inline
        if (amount <= 0) return "ERROR_AMOUNT";
        if (userId == null || userId.isEmpty()) return "ERROR_USER";
        if (!Arrays.asList("WAVE", "OM", "CARD").contains(type))
            return "ERROR_TYPE";

        // Logique métier + Infrastructure mélangées
        Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
        PreparedStatement ps = conn.prepareStatement(
            "SELECT balance FROM users WHERE id = '" + userId + "'"); // SQL INJECTION!
        ResultSet rs = ps.executeQuery();
        double balance = rs.getDouble("balance");

        if (balance < amount) return "ERROR_INSUFFICIENT_FUNDS";

        // Appel externe inline, pas de timeout, pas de retry propre —
        URL url = new URL("http://payment-gateway/api/" + type + "/charge");
        HttpURLConnection con = (HttpURLConnection) url.openConnection();
        // ... 200 lignes de gestion HTTP manuelle ...

        // Email inline avec credentials hardcodés
        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.company.com");
        props.put("mail.smtp.password", "P@ssw0rd123"); // CREDENTIAL EN DUR
        // ...
        return transactionId;
    }
}

// =====
// PHASE 2 : HARNAIS DE TEST — Sans modifier le code legacy
// Créer une interface Seam pour intercepter les dépendances externes
// =====

// Interface extraite mécaniquement (refactoring IDE — pas de logique)
interface UserRepository {
    Optional<User> findById(String userId);
}

interface PaymentGateway {
    GatewayResponse charge(ChargeRequest request);
}

interface NotificationService {
    void sendConfirmation(String userId, String transactionId);
}
```

```

// Test rendu possible par injection des seams —————
@Test
void should_return_error_when_insufficient_funds() {
    // Arrange : Mock des dépendances externes
    UserRepository fakeRepo = userId -> Optional.of(
        new User(userId, new BigDecimal("500"))); // Solde insuffisant
    PaymentGateway fakeGateway = req -> GatewayResponse.success("txn_001");

    PaymentProcessor processor =
        new PaymentProcessor(fakeRepo, fakeGateway, mock(NotificationService.class));

    // Act
    PaymentResult result = processor.processPayment(
        CreatePaymentCommand.of("user_1", new BigDecimal("1000"), "WAVE"));

    // Assert
    assertThat(result.status()).isEqualTo(PaymentStatus.INSUFFICIENT_FUNDS);
    verify(fakeGateway, never()).charge(any()); // Gateway jamais appelé
}

// =====
// PHASE 3 : REFACTORING INCRÉMENTAL sous couverture de test
// =====

// Étape 3a : Extract Method – Validation isolée et testable ———
public class PaymentValidator {
    public ValidationResult validate(CreatePaymentCommand cmd) {
        List<String> errors = new ArrayList<>();
        if (cmd.amount().compareTo(BigDecimal.ZERO) <= 0)
            errors.add("Amount must be positive");
        if (cmd.amount().compareTo(MAX_TRANSFER) > 0)
            errors.add("Amount exceeds limit of " + MAX_TRANSFER + " XOF");
        if (cmd.userId() == null || cmd.userId().isBlank())
            errors.add("UserId is required");
        return errors.isEmpty() ? ValidationResult.valid()
            : ValidationResult.invalid(errors);
    }
}

// Étape 3b : Introduce Value Object – Plus de primitives en paramètres
public record CreatePaymentCommand(
    UserId        userId,
    TransferAmount amount, // Validation encapsulée dans le VO
    PaymentMethod method,
    Currency       currency
) {
    // Validation dans le constructeur – impossible de créer un state invalide
    public CreatePaymentCommand {
        Objects.requireNonNull(userId, "userId required");
        Objects.requireNonNull(amount, "amount required");
        Objects.requireNonNull(method, "method required");
        Objects.requireNonNull(currency, "currency required");
    }
}

// =====
// PHASE 4 : STRANGLER FIG – Remplacement progressif en production
// =====

// Façade qui route entre legacy et nouveau code selon feature flag

```

```
@Service
public class PaymentProcessorFacade {
    private final LegacyPaymentProcessor legacyProcessor;
    private final ModernPaymentProcessor modernProcessor;
    private final FeatureFlags flags;

    public PaymentResult processPayment(CreatePaymentCommand cmd) {
        if (flags.isEnabled("modern-payment-processor", cmd.userId().value())) {
            // Nouveau code : 10% des users → 50% → 100% progressivement
            return modernProcessor.process(cmd);
        }
        // Legacy : toujours disponible comme fallback
        return legacyProcessor.processPayment(
            cmd.userId().value(), cmd.amount().value().doubleValue(),
            cmd.method().name(), cmd.currency().code(), false, 3
        );
    }
}

// Déploiement : 0% modern → 1% → 5% → 20% → 50% → 100%
// Rollback immédiat si anomalie : flags.disable('modern-payment-processor')
```

◆ Métriques de Qualité Code : Mesurer le Progrès

Le refactoring sans métriques est une démarche artistique. Avec des métriques, c'est de l'ingénierie. Voici les indicateurs objectifs qui mesurent la progression de votre base de code vers l'excellence :

Métrique	Outil	Seuil Acceptable	Seuil Alarme	Signification
Cyclomatic Complexity	SonarQube / PMD	< 10 / méthode	> 15	Complexité des branches
Test Coverage	JaCoCo / Istanbul	> 80%	< 60%	% code couvert par tests
Code Duplication	SonarQube	< 3%	> 10%	DRY violations
Cognitive Complexity	SonarQube	< 15 / méthode	> 25	Difficulté de compréhension
Method Length	CheckStyle	< 30 lignes	> 50 lignes	SRP violation probable
Afferent Coupling (Ca)	JDepend	< 10	> 20	Classe trop utilisée = risque
Efferent Coupling (Ce)	JDepend	< 10	> 15	Classe trop dépendante
Instability (I=Ce/Ca+Ce)	JDepend	0.2 - 0.8	= 0 ou = 1	Classes centrales vs feuilles


```

* MAVEN / YAML / JAVA - QUALITY GATE CI/CD + ARCHUNIT TESTS

# — CI/CD Quality Gate avec SonarQube + Maven —————
# sonar-project.properties
sonar.projectKey=arsenal-payment-service
sonar.projectName=Arsenal Payment Service
sonar.sources=src/main/java
sonar.tests=src/test/java
sonar.java.binaries=target/classes
sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml

# Quality Gate personnalisé (bloque le merge si violé) :
# — Conditions définies dans SonarQube UI —————
# Coverage on New Code < 80% → FAILED
# Duplicated Lines on New Code > 3% → FAILED
# Maintainability Rating on New Code worse C → FAILED
# Reliability Rating on New Code worse B → FAILED
# Security Hotspots Reviewed < 100% → FAILED

# — Pipeline GitHub Actions —————
name: Quality Gate
on: [pull_request]
jobs:
  sonar-analysis:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Historique complet pour analyse de diff

      - name: Run Tests + Coverage
        run: mvn test jacoco:report

      - name: SonarQube Analysis
        run: mvn sonar:sonar
          -Dsonar.host.url=${{ secrets.SONAR_URL }}
          -Dsonar.login=${{ secrets.SONAR_TOKEN }}

      - name: Quality Gate Check
        uses: sonarsource/sonarqube-quality-gate-action@v1.1.0
        with:
          scanMetadataReportFile: target/sonar/report-task.txt
          # Bloque automatiquement la PR si Quality Gate échoue

# — ArchUnit : Tests d'architecture automatisés (dans la CI) —————
@AnalyzeClasses(packages = "com.arsenal.payment")
public class ArchitectureTest {

  @ArchTest
  static final ArchRule services_should_not_depend_on_controllers =
    noClasses().that().resideInAPackage("..service..")
      .should().dependOnClassesThat()
      .resideInAPackage("..controller..");

  @ArchTest
  static final ArchRule repositories_only_in_infrastructure =
    classes().that().haveNameMatching(".*Repository")
      .should().resideInAPackage("..infrastructure..");

  @ArchTest
  static final ArchRule domain_has_no_framework_dependencies =

```

```
noClasses().that().resideInAPackage("..domain..")
    .should().dependOnClassesThat()
        .resideInAnyPackage("org.springframework..","javax.persistence..");
// Le domaine ne doit JAMAIS connaître Spring ou JPA
}
```

Synthèse Finale : L'Arsenal Tech est Complet

Vous avez parcouru les cinq chapitres qui définissent l'ingénieur élite en 2026. De l'architecture microservices à Kubernetes, de la souveraineté numérique africaine aux pipelines de données God-Mode, de l'IA générative en production au Clean Code militaire — ce n'est pas une liste de technologies à connaître. C'est un système de pensée à adopter.

L'ingénieur d'élite n'est pas celui qui connaît le plus d'outils. C'est celui qui voit les systèmes en entier : qui comprend pourquoi Cassandra est structurellement incapable de garantir des transactions ACID, pourquoi un agent autonome sans garde-fous est un risque légal, pourquoi le refactoring sans tests préalables est une faute professionnelle. La connaissance technique sans le discernement architectural n'est que de la virtuosité sans direction.

LES 10 COMMANDEMENTS DE L'INGÉNIEUR ARSENAL TECH 2026

- I. Tu architectes selon les contraintes de domaine, pas selon la dernière tendance LinkedIn.
- II. Tu modélises Cassandra selon tes queries, MongoDB selon tes documents, Neo4j selon tes relations.
- III. Tu ne déploies jamais un LLM sans monitoring de faithfulness et quality gate RAGAS.
- IV. Tu n'écrites jamais un agent autonome sans human-in-the-loop sur les actions irréversibles.
- V. Tu respecteras le CAP theorem — il n'est pas négociable, même sous pression du management.
- VI. Tu ajouteras des tests AVANT de refactorer le legacy, jamais après.
- VII. Tu ne violeras pas l'OCP — ajouter un type de paiement ne modifie JAMAIS du code existant.
- VIII. Tu chiffreras côté client avant tout transfer vers le cloud — la souveraineté est une archi.
- IX. Tu monitors tes pipelines Spark avec AQE activé et tu partitionnes selon tes patterns de query.
- X. Tu écriras du code comme s'il allait être lu en production à 3h du matin sous incident critique.

Partie	Chapitre	Compétence Clé Acquise	Outil Maître
I — Architecture	1 — Microservices	Event-driven à 10M TPS	K8s + Kafka + Istio
I — Architecture	2 — Souveraineté	Cloud Hybride UEMOA conforme	Terraform + AES-256-GCM
II — Data	3 — Pipelines	Lakehouse Bronze/Silver/Gold	Spark AQE + Iceberg + Cassandra
II — Data	4 — IA Générative	LLMOps + Agent + ReAct/CoV	LangGraph + RAGAS + DSPy
III — Engineering	5 — Clean Code	SOLID + GoF + Refactoring Legacy	SonarQube + ArchUnit + JaCoCo

ARSENAL TECH — L'ÉLITE 2026

5 Chapitres · 3 Parties · 100 Pages
Architecture · Data Engineering · Software Engineering

